

使用 ML Kit 辨識文字和臉部特徵：iOS

來源：<https://codelabs.developers.google.com/codelabs/mlkit-ios?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將使用 ML Kit 建構 iOS 應用程式，這個套件使用裝置端機器學習技術來辨識圖片中的文字和臉部特徵。

目錄

1. [1. 簡介](#)
2. [2. 開始設定](#)
3. [3. 執行範例應用程式](#)
4. [4. 新增裝置端文字辨識功能](#)
5. [5. 新增裝置端臉部輪廓偵測功能](#)
6. [6. 恭喜！](#)

1. 簡介

ML Kit 是一套行動 SDK，可將 Google 的機器學習專業知識導入 Android 和 iOS 應用程式，功能強大且易於使用。無論您是機器學習新手還是經驗豐富的開發人員，只需幾行程式碼，就能輕鬆導入所需功能。您不需要深入瞭解類神經網路或模型最佳化，即可開始使用。

運作方式

ML Kit 將 Google 的機器學習技術 (例如 [Mobile Vision](#) 和 [TensorFlow Lite](#)) 整合至單一 SDK 中，方便您輕鬆將機器學習技術運用在自家應用程式中。無論您需要 Mobile Vision 裝置端模型提供的即時功能，還是自訂 TensorFlow Lite 圖片分類模型的彈性，ML Kit 都能讓您輕鬆達成，只需要幾行程式碼即可。

本程式碼實驗室將逐步說明如何建立自己的 iOS 應用程式，自動偵測圖片中的文字和臉部特徵。

您是 Android 開發人員嗎？請參閱本程式碼研究室的 [Android 特定版本](#)。

建構目標

在本程式碼研究室中，您將使用 [ML Kit](#) 建構 iOS 應用程式。您的應用程式將會：

- 使用 ML Kit Text Recognition API 偵測圖片中的文字
- 使用 ML Kit 臉部偵測 API 識別圖片中的臉部特徵



課程內容

- 如何使用 ML Kit SDK 輕鬆為任何 iOS 應用程式新增進階機器學習功能，例如文字辨識、臉部特徵偵測

軟硬體需求

- 最新版 Xcode (12.4 以上)
- iOS 模擬器或搭載 iOS 10.0 以上版本的實體 iOS 裝置
- ML Kit 僅支援這兩種 64 位元架構：`x86_64` 和 `arm64`
- [程式碼範例](#)
- 具備 Swift 的 iOS 開發基本知識
- 對機器學習模型有基本瞭解

本程式碼研究室著重於 ML Kit。我們不會對與主題無關的概念和程式碼多做介紹，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

步驟 2 · 約 1 分鐘

2. 開始設定

下載程式碼

點選下方連結即可下載這個程式碼研究室的所有程式碼：

將下載的 ZIP 檔案解壓縮。這會建立根資料夾 (`mlkit-ios-codelab`)，其中包含您需要的所有資源。在本程式碼研究室中，您只需要 `vision` 子目錄中的資源。

`mlkit-ios-codelab` 存放區中的 `vision` 子目錄包含兩個目錄：



- **starter**：本程式碼研究室的範例程式碼。



- **final**：完成的範例應用程式程式碼。

使用 CocoaPods 新增 ML Kit 的依附元件

CocoaPods 用於將 ML Kit 依附元件新增至應用程式。如果您的電腦尚未安裝 CocoaPods，請參閱[這篇文章](#)瞭解安裝操作說明。安裝完成後，請在偏好的編輯器中開啟 Podfile，並將 ML Kit 新增為依附元件：

Podfile

```
platform :ios, '10.0'
use_frameworks!

pod 'GoogleMLKit/FaceDetection'
pod 'GoogleMLKit/TextRecognition'

target 'MLKit-codelab' do
end
```

安裝 ML Kit Cocoa Pod

為確保應用程式可使用所有依附元件，請使用指令列安裝 ML Kit Cocoa Pod。

指令列

```
# Make sure you are in the root of your app
pod install
xed .
```

步驟 3 · 約 1 分鐘

3. 執行範例應用程式

現在可以首次執行應用程式。在 Xcode 中按一下「Run」



，編譯應用程式並在 iOS 模擬器上執行。

應用程式應會在模擬器上啟動。此時您應該會看到基本版面配置，其中包含可供選取 2 張圖片的挑選器。在下一節中，您將在應用程式中新增文字辨識功能，識別圖片中的文字。

步驟 4 · 約 5 分鐘

4. 新增裝置端文字辨識功能

在這個步驟中，我們會在應用程式中新增功能，辨識圖片中的文字。

匯入 MLVision 模組

確認 `ViewController` 類別中存在下列匯入項目。

ViewController.swift

```
import MLKit
```

建立 VisionTextRecognizer

在 `ViewController` 類別中新增下列延遲屬性。

ViewController.swift

```
private lazy var textRecognizer = TextRecognizer.textRecognizer()
```

設定及執行圖片的裝置端文字辨識

在 `ViewController` 類別的 `runTextRecognition` 方法中新增下列內容：

ViewController.swift

```
func runTextRecognition(with image: UIImage) {  
    let visionImage = VisionImage(image: image)  
    textRecognizer.process(visionImage) { features, error in  
        self.processResult(from: features, error: error)  
    }  
}
```

上述程式碼會設定文字辨識偵測器，並使用回應呼叫 `processResult(from:, error:)` 函式。

處理文字辨識回應

在 `ViewController` 類別中將下列程式碼新增至 `processResult`，即可剖析結果並顯示在應用程式中。

ViewController.swift

```

func processResult(from text: Text?, error: Error?) {
    removeDetectionAnnotations()
    guard error == nil, let text = text else {
        let errorString = error?.localizedDescription ?? Constants.detectionNoResultsMessage
        print("Text recognizer failed with error: \(errorString)")
        return
    }

    let transform = self.transformMatrix()

    // Blocks.
    for block in text.blocks {
        drawFrame(block.frame, in: .purple, transform: transform)

        // Lines.
        for line in block.lines {
            drawFrame(line.frame, in: .orange, transform: transform)

            // Elements.
            for element in line.elements {
                drawFrame(element.frame, in: .green, transform: transform)

                let transformedRect = element.frame.applying(transform)
                let label = UILabel(frame: transformedRect)
                label.text = element.text
                label.adjustsFontSizeToFitWidth = true
                self.annotationOverlayView.addSubview(label)
            }
        }
    }
}

```

在模擬器上執行應用程式

現在請在 Xcode 中點選「執行」



。應用程式載入後，請確認選取器中已選取 **Image 1**，然後按一下 **Find Text** 按鈕。

現在，應用程式看起來會如下圖所示，在原始圖片上疊加顯示文字辨識結果和周框。

ML Kit



Find Text

Find Face Contour

相片：Kai Schreiber / Wikimedia Commons / CC BY-SA 2.0

恭喜！您已使用 ML Kit，在應用程式中新增裝置端文字辨識功能。裝置端文字辨識功能適用於許多用途，即使應用程式沒有網路連線也能運作，而且速度夠快，可辨識靜態圖片和即時影像畫面中的文字。

步驟 5 · 約 5 分鐘

5. 新增裝置端臉部輪廓偵測功能

在這個步驟中，我們會在應用程式中新增功能，偵測圖片中的臉部輪廓。

建立 FaceDetector

在 `ViewController` 類別中新增下列延遲屬性。

ViewController.swift

```
private lazy var faceDetectorOption: FaceDetectorOptions = {
    let option = FaceDetectorOptions()
    option.contourMode = .all
    option.performanceMode = .fast
    return option
}()
private lazy var faceDetector = FaceDetector.faceDetector(options: faceDetectorOption)
```

在圖片上設定及執行裝置端臉部輪廓偵測

在 `ViewController` 類別的 `runFaceContourDetection` 方法中新增下列內容：

ViewController.swift

```
func runFaceContourDetection(with image: UIImage) {
    let visionImage = VisionImage(image: image)
    faceDetector.process(visionImage) { features, error in
        self.processResult(from: features, error: error)
    }
}
```

上述程式碼會設定文字辨識偵測器，並使用回應呼叫 `processResult(from:, error:)` 函式。

處理臉部偵測器回應

在 `ViewController` 類別中將下列程式碼新增至 `processResult`，即可剖析結果並顯示在應用程式中。

ViewController.swift

```
func processResult(from faces: [Face]?, error: Error?) {
    removeDetectionAnnotations()
    guard let faces = faces else {
        return
    }

    for feature in faces {
        let transform = self.transformMatrix()
        let transformedRect = feature.frame.applying(transform)
        UIUtilities.addRectangle(
            transformedRect,
            to: self.annotationOverlayView,
            color: UIColor.green
        )
        self.addContours(forFace: feature, transform: transform)
    }
}
```

最後，在 `ViewController` 類別中新增輔助方法 `addContours`，繪製輪廓點。

ViewController.swift

```

private func addContours(forFace face: Face, transform: CGAffineTransform) {
    // Face
    if let faceContour = face.contour(ofType: .face) {
        for point in faceContour.points {
            drawPoint(point, in: .blue, transform: transform)
        }
    }

    // Eyebrows
    if let topLeftEyebrowContour = face.contour(ofType: .leftEyebrowTop) {
        for point in topLeftEyebrowContour.points {
            drawPoint(point, in: .orange, transform: transform)
        }
    }
    if let bottomLeftEyebrowContour = face.contour(ofType: .leftEyebrowBottom) {
        for point in bottomLeftEyebrowContour.points {
            drawPoint(point, in: .orange, transform: transform)
        }
    }
    if let topRightEyebrowContour = face.contour(ofType: .rightEyebrowTop) {
        for point in topRightEyebrowContour.points {
            drawPoint(point, in: .orange, transform: transform)
        }
    }
    if let bottomRightEyebrowContour = face.contour(ofType: .rightEyebrowBottom) {
        for point in bottomRightEyebrowContour.points {
            drawPoint(point, in: .orange, transform: transform)
        }
    }

    // Eyes
    if let leftEyeContour = face.contour(ofType: .leftEye) {
        for point in leftEyeContour.points {
            drawPoint(point, in: .cyan, transform: transform)
        }
    }
    if let rightEyeContour = face.contour(ofType: .rightEye) {
        for point in rightEyeContour.points {
            drawPoint(point, in: .cyan, transform: transform)
        }
    }

    // Lips
    if let topUpperLipContour = face.contour(ofType: .upperLipTop) {
        for point in topUpperLipContour.points {
            drawPoint(point, in: .red, transform: transform)
        }
    }
    if let bottomUpperLipContour = face.contour(ofType: .upperLipBottom) {
        for point in bottomUpperLipContour.points {
            drawPoint(point, in: .red, transform: transform)
        }
    }
}

```

```

    }
    if let topLowerLipContour = face.contour(ofType: .lowerLipTop) {
        for point in topLowerLipContour.points {
            drawPoint(point, in: .red, transform: transform)
        }
    }
    if let bottomLowerLipContour = face.contour(ofType: .lowerLipBottom) {
        for point in bottomLowerLipContour.points {
            drawPoint(point, in: .red, transform: transform)
        }
    }

    // Nose
    if let noseBridgeContour = face.contour(ofType: .noseBridge) {
        for point in noseBridgeContour.points {
            drawPoint(point, in: .yellow, transform: transform)
        }
    }
    if let noseBottomContour = face.contour(ofType: .noseBottom) {
        for point in noseBottomContour.points {
            drawPoint(point, in: .yellow, transform: transform)
        }
    }
}

```

在模擬器上執行應用程式

現在請在 Xcode 中點選「執行」



。應用程式載入後，請確認選取器中已選取 `Image 2`，然後按一下 `Find Face Contour` 按鈕。現在，您的應用程式看起來會如下圖所示，顯示葛麗絲·霍普臉部的輪廓，以點的形式疊加在原始圖片上。



恭喜！您剛才使用裝置端 ML Kit，在應用程式中新增了裝置端臉部輪廓偵測功能。裝置端 ML Kit 臉部輪廓偵測功能適用於許多用途，即使應用程式沒有網際網路連線也能運作，而且速度夠快，可處理靜態圖片和即時影像影格。

步驟 6 · 約 0 分鐘

6. 恭喜！

您已使用 ML Kit 輕鬆為應用程式新增進階機器學習功能。

涵蓋內容

- 如何將 ML Kit 新增至 iOS 應用程式
- 如何使用 ML Kit 的裝置端文字辨識功能，找出圖片中的文字
- 如何使用 ML Kit 的裝置端臉部辨識功能，辨識圖片中的臉部特徵

後續步驟

- 在自己的 iOS 應用程式中使用 ML Kit。

瞭解詳情

- <https://g.co/mlkit>
-